

Premise Selection for Mathematics by Corpus Analysis and Kernel Methods

Jesse Alama · Tom Heskes · Daniel Kühlwein ·
Evgeni Tsivtsivadze · Josef Urban

Received: 10 August 2011 / Accepted: 2 April 2013 / Published online: 24 April 2013
© Springer Science+Business Media Dordrecht 2013

Abstract Smart premise selection is essential when using automated reasoning as a tool for large-theory formal proof development. This work develops learning-based premise selection in two ways. First, a fine-grained dependency analysis of existing high-level formal mathematical proofs is used to build a large knowledge base of proof dependencies, providing precise data for ATP-based re-verification and for training premise selection algorithms. Second, a new machine learning algorithm for premise selection based on kernel methods is proposed and implemented. To evaluate the impact of both techniques, a benchmark consisting of 2078 large-theory mathematical problems is constructed, extending the older MPTP Challenge benchmark. The combined effect of the techniques results in a 50 % improvement on the benchmark over the state-of-the-art Vampire/SInE system for automated reasoning in large theories.

Keywords Automated reasoning in large theories · Machine learning · Premise selection · Automated theorem proving

Research work of Jesse Alama was funded by FCT project “Dialogical Foundations of Semantics” (DiFoS) in the ESF EuroCoRes programme LogICCC (FCT LogICCC/0001/2007). Research for this paper was partially done while a visiting fellow at the Isaac Newton Institute for the Mathematical Sciences in the programme ‘Semantics & Syntax’.

Research works of T. Heskes, D. Kühlwein, E. Tsivtsivadze and J. Urban were funded by the NWO projects “Learning2Reason” and “MathWiki”.

J. Alama

Center for Artificial Intelligence, New University of Lisbon, 1099085 Lisbon, Portugal

T. Heskes · D. Kühlwein · E. Tsivtsivadze · J. Urban (✉)
Intelligent Systems, Institute for Computing and Information Sciences,
Radboud University, Nijmegen, Netherlands
e-mail: Josef.Urban@gmail.com

1 Introduction

In this paper we significantly improve theorem proving over large formal mathematical libraries by using a two-phase approach combining precise proof analysis with machine learning algorithms for premise selection.

The first phase uses a newly available *fine-grained dependency analysis* [4] of the proofs in the large Mizar Mathematical Library (MML).¹ This analysis allows us to construct precise problems for ATP-based re-verification of the Mizar proofs. More importantly, the precise dependency data can be used as a training and testing corpus for premise selection algorithms.

In the second phase, a complementary improvement is achieved by using a new kernel-based machine learning algorithm, which outperforms existing methods for premise selection. This means that based on the large number of previously solved mathematical problems, we can more accurately estimate which premises will be useful for proving a new conjecture.

Such learned knowledge considerably helps automated proving of new formally expressed mathematical problems by recommending the most relevant previous theorems and definitions from the very large existing libraries, and thus shielding the existing ATP methods from considering thousands of irrelevant axioms. The better such symbiosis of formal mathematics and learning-assisted automated reasoning gets, the better for both parties: improved automated reasoning increases the efficiency of formal mathematicians, and lowers the cost of producing formal mathematics. This in turn leads to larger corpora of previously solved nontrivial problems from which the learning-assisted ATP can extract additional problem-solving knowledge covering larger and larger parts of mathematics.

The rest of the paper is organized as follows. Section 2 describes recent developments in large-theory automated reasoning and provides motivation for the investigation. Section 3 summarizes the recent implementation of fine-grained dependency analysis over the large MML, and its use for ATP-based cross-verification and training premise selection. Section 4 describes the general machine learning approach to premise selection and an efficient kernel-based multi-output ranking algorithm for premise selection. In Section 5 a new large-theory benchmark of 2078 related MML problems is defined, extending the older and smaller MPTP Challenge [34] benchmark, and our techniques are evaluated on this benchmark in Section 6. Section 7 concludes and discusses future work and extensions.

2 Automated Reasoning in Large Theories (ARLT)

In recent years, large formal libraries of re-usable knowledge expressed in rich formalisms have been built with interactive proof assistants, such as Mizar [11], Isabelle [17], Coq [6], HOL (light) [13], and others. Formal approaches are also being used increasingly in non-mathematical fields such as software and hardware verification and common-sense reasoning about real-world knowledge. Such trends lead to growth of formal knowledge bases in these fields.

¹<http://www.mizar.org>

One important development is that a number of these formal knowledge bases and core logics have been translated to first-order formats suitable for ATPs [15, 19, 35], and first-order ATPs such as Vampire [20], E [26], SPASS [40], and recently also SMT (satisfiability modulo theories) solvers like Z3 [16] are today routinely used for proof assistance in systems like Isabelle [8, 18] and Mizar [37, 38]. These first-order translations give rise to large, semantically rich corpora that present significant challenges for the field of automated reasoning. One particular problem is the size of the problems generated by these translations. ATPs struggle to cope with large numbers of (often) unnecessary premises. To improve the performance of ATPs, it is essential to limit the set of available premises to the most relevant ones. Such premise-selection techniques developed so far can be broadly divided into two categories:

1. Heuristic symbolic analysis of the formulas appearing in problems, and
2. Analysis of previous proofs.

In the first category, Hoder's SInE preprocessor [14] has so far been the most successful. SInE is particularly strong in domains with many hierarchical definitions such as those in common-sense ontologies. In the second category, machine learning of premise selection, as done e.g. by the MaLARea [39] system, is an effective method in hard mathematical domains, where the knowledge bases contain proportionally many more nontrivial lemmas and theorems than simple definitions, and previous verified proofs can be used for learning proof guidance.

Automated reasoning in large mathematical corpora is an interesting field in several respects. Large theories permit data-driven approaches [28] to constructing ATP algorithms; indeed, the sheer size (tens of thousands of definitions and theorems) of such libraries actually necessitates such methods. It turns out that purely deductive, brute-force search methods can be improved significantly by heuristic and inductive² methods, thus allowing experimental research into combinations [39] of inductive and deductive methods. Large-theory benchmarks like the MPTP Challenge,³ and its extended version developed here in Section 5, can serve for rigorous evaluation of such novel Artificial Intelligence (AI) methods over thousands of mathematical problems. Apart from the *novel AI* aspect, and the obvious *proof assistance* aspect, automated reasoning over large formal mathematical corpora can also become a new tool in the established field of *reverse mathematics* [29]. This line of research has been already started, for example by Solovay's analysis [30] of the connection between Tarski's axiom [31] and the axiom of choice, and by Alama's analysis of the Euler's polyhedron formula [1], both conducted over the MML.

3 Computing Dependencies in Mizar

The Mizar Mathematical Library (MML) is one of the largest corpora of formalized mathematical knowledge [11]. The theorems of the MML are proved in a declarative natural deduction-style formalism based on first-order classical logic and Tarski-Grothendieck set theory [32]. The MML is divided into "articles" which, like an article

²The word *inductive* denotes here *inductive reasoning*, as opposed to *deductive reasoning*.

³<http://www.tptp.org/MPTPChallenge>

in a mathematics journal, are unified collections of definitions, theorems, and proofs. At the time of writing, the MML contains more than 50,000 theorems and more than 10,000 definitions, divided into more than 1,100 articles.⁴

In the world of automated theorem proving, proofs contain essentially all logical steps, even very small ones (such as the steps taken in a resolution proof). In the world of interactive theorem proving, one of the goals is to allow the users to express themselves with relatively less verbosity. Towards that end, interactive theorem proving (ITP) systems often come with mechanisms for suppressing some steps of an argument. This means that often it is not explicitly visible what a particular proof depends upon.

Already in the MPTP [35] system, a method (denoted D below) for computing a set of dependencies is implemented. This MPTP heuristic starts by taking all *explicit* premises contained in the original human-written Mizar proof. To get all the premises used by Mizar *implicitly*, the heuristic watches the problem's set of symbols, and adds the implicitly used formulas (typically typing formulas about the problem's symbols) in a fixpoint manner. The heuristic attempts (by considering all kinds of possible Mizar mechanisms) to guarantee completeness, but the dependencies it finds are often still quite redundant

In the experiments below, we use a finer method [4] (denoted FD) to identify the dependencies of Mizar items. The first step in the computation of such fine-grained dependencies in Mizar is to break up each article in the MML into a sequence of Mizar texts, each consisting of a single top-level item (e.g., a theorem, definition, unexported lemma). Each of these texts can—with suitable preprocessing—be regarded as a complete, valid Mizar article in its own right. The decomposition of a whole MML article into such smaller articles typically requires a number of nontrivial refactoring steps, comparable, e.g., to automated splitting and re-factoring of large programs written in programming languages with complicated syntactic mechanisms.

In Mizar, every article has a so-called *environment*: a list $ENV_0 = [\text{item}_j : 1 \leq j \leq \text{length}(ENV_0)]$ of items item_j specifying the background knowledge (theorems, notations, etc.) that is used to verify the article. The actual Mizar content contained in an article's environment, is, in general, a rather conservative overestimate of the items that the article actually needs. The FD method first defines the current environment as ENV_0 . It then considers each item in ENV_0 and tries to verify the article using the current environment without the considered item. If the verification succeeds, the considered item is deleted from the current environment. To be more precise, starting with the original environment ENV_0 (in which the article verification succeeds), the FD method works by constructing for the article a sequence of finer environments $\{ENV_i : 1 \leq i \leq \text{length}(ENV_0)\}$ such that

$$ENV_i := \begin{cases} ENV_{i-1} & \text{if the article verification fails in } ENV_{i-1} - \{\text{item}_i\} \\ ENV_{i-1} - \{\text{item}_i\} & \text{otherwise.} \end{cases}$$

The article verification thus still succeeds in the final $ENV_{\text{length}(ENV_0)}$ environment, and this environment consist of all the items of ENV_0 whose removal caused the article verification to fail during this construction. This final environment could in general still be made smaller (after the removal of a certain item, another item might

⁴The experiments were conducted with Mizar system version 7.11.07 and MML version 4.160.1126.

become unnecessary), and its construction depends on the (chosen and fixed for all experiments) initial ordering of items in the environment. Given this fixed initial ordering, the *FD* method proceeds deterministically and results in a typically much smaller set of dependencies (compared to method *D*) for a given Mizar item. The drawback of this method is that such exhaustive approach can be time consuming.⁵ The advantage is that this method removes many redundant dependencies which are typically drawn in by overly powerful proof checking algorithms (like congruence closure over sets of all available equalities, etc.) when the dependency tracking is implemented internally inside a proof assistant. Such dependency pruning is particularly important for the ATP and premise-selection applications that are explained in this paper: spending a day to compute finer dependencies is a very good time investment if it can provide better guidance for the fast-growing search space explored by ATPs. Another advantage of this approach is that it also provides *syntactic* dependencies, which are needed for real-time recompilation of the particular item as written in the article. This functionality is important for fast fine-grained recompilation in formal wikis [2], however for semantic applications like ATP we are only considering the truly semantic dependencies, i.e., those dependencies that result in a formula when translated by the MPTP system [35] to first-order logic.

4 Premise Selection in Large Theories by Machine Learning

When reasoning over a large theory (like the *MML*), thousands of premises are available. In the presence of such large numbers of premises, the performance of most ATP systems degrades considerably [36]. Yet typically only a fraction of the available premises are actually needed to construct a proof. Estimating which premises are likely to be useful for constructing a proof is our research problem:

Definition 1 (Premise selection problem) Given an ATP *A*, a large number of premises $\mathcal{P}$ and a new conjecture *c*, predict those premises from $\mathcal{P}$ that are likely to be useful for *A* for constructing a proof of *c*.

Knowledge of previous proofs and problem-solving techniques is used by mathematicians to guide their thinking about new problems. The detailed *MML* proof analysis described above provides a large computer-understandable corpus of dependencies of mathematical proofs. This information can be employed to train learning algorithms as to which premises are useful given a new conjecture. In this section we present the machine learning setting and algorithms that are used for premise selection on such corpora. Our goal is to begin emulating the training of human mathematicians.

Note that in the *MML*, there is always exactly one (typically a textbook) proof of a particular theorem *c*, and hence exactly one set of premises used in the proof of *c*. This corresponds to the mathematical textbook practice where typically only

⁵This can be improved by heuristics for guessing the needed dependencies, analogous to those used for ATP premise selection [4].

one proof is given for a particular theorem. Investigating the effect of having several different proofs for the same statement is a possible extension of this work.⁶

Furthermore we should note that we are using MML proofs to learn premise selection for an ATP. The underlying assumption is that the MML dependencies are also useful as ATP dependencies, and indeed as we shall see in Section 6 this is (at least to some extent) the case. It might be (depending on the ATP implementation) that a fully automatically found proof would provide better training data than a MML proof. A major obstacle for such training is however the relative weakness of existing ATPs in finding more involved proofs of MML theorems [36], and thus their failure to provide the training examples for a large part of MML. Still, a comparison of the power of training on MML and ATP proofs could be interesting future work.

When the translation [35] from Mizar to ATP formats is applied, the Mizar theorems and their proof dependencies (definitions, other theorems, etc.) translate to first-order formulas, used in the corresponding ATP problems as *conjectures* and their *premises* (axioms). For further presentation here we identify each MML formula with its first-order translation.⁷ We will work in the following setting, which is tailored to the MML, but can easily be translated to other large datasets. Let Γ be the set of first order formulas that appear in the MML.

Definition 2 (Proof matrix) Using the fine-grained Mizar proof analysis—which says for each pair of formulas $p, c \in \Gamma$ whether p is used in the MML proof of c - define the function $\mu : \Gamma \times \Gamma \rightarrow \{0, 1\}$ by

$$\mu(c, p) := \begin{cases} 1 & \text{if } p \text{ is used to prove } c, \\ 0 & \text{otherwise.} \end{cases}$$

In other words, μ is the adjacency matrix of the graph of the fine-grained MML proof dependencies. This proof matrix, together with suitably chosen formula features, will be used for training machine learning algorithms. The used premises of a conjecture c are denoted by

$$\text{usedPremises}(c) := \{p \mid \mu(c, p) = 1\}$$

Definition 3 (Feature matrix) We characterize MML formulas by the symbols⁸ and (sub)terms appearing in them. We use de Bruijn indices for variables, and term equality is then just string equality. Let $T := \{t_1, \dots, t_m\}$ be a fixed enumeration of the set of all symbols and (sub)terms that appear in all formulas from Γ .⁹ We define $\Phi : \Gamma \times \{1, \dots, m\} \rightarrow \{0, 1\}$ by

$$\Phi(c, i) := \begin{cases} 1 & \text{if } t_i \text{ appears in } c, \\ 0 & \text{otherwise.} \end{cases}$$

⁶For example, one could image to learn from a mixture of MML and ATP proofs.

⁷Mizar is “nearly first-order”, so the correspondence is “nearly” one-to-one. In particular it is possible to construct the set of ATP premises from the fine-grained Mizar dependencies for each Mizar theorem.

⁸Logical symbols like $\wedge, \vee$ are ignored since they appear in almost every formula.

⁹If the set of features is not constant they are enumerated in order of appearance.

This matrix gives rise to the *feature function* $\varphi : \Gamma \rightarrow \{0, 1\}^m$ which for $c \in \Gamma$ is the vector φ^c with entries in $\{0, 1\}$ satisfying

$$\varphi_i^c = 1 \iff \Phi(c, i) = 1.$$

The *expressed features of a formula* are denoted by the value of the function $e : \Gamma \rightarrow \mathcal{P}(T)$ that maps c to $\{t_i \mid \Phi(c, i) = 1\}$.

Different features (such as only symbols, or only subterms) could be used. The better the features correspond to the concepts that are relevant when choosing theorems for solving a particular problem, the more successful the machine learning of premise selection can be. The particular heuristic justification of using formula (sub)terms is given in [39].

The premise selection problem can be treated as a ranking problem, or as a classification problem. In the ranking approach, we for a given a conjecture c rank the available premises by their predicted usefulness for an automated proof of c , and use some number n of premises with the highest ranking (denoted here as *advisedPremises*(c, n)). In the classification approach, we are looking for each premise $p \in \Gamma$ for a real-valued *classifier function* $C_p(\cdot) : \Gamma \rightarrow \mathbb{R}$ which, given a conjecture c , estimates how useful p is for proving c . In standard classification, a premise p would then be used if $C_p(c)$ is above certain threshold. A common approach to ranking is to use classification, and to combine the real-valued classifiers [21]: the premises for a conjecture c are ranked by the values of $C_p(c)$, and we choose a certain number of the best ones. This is the approach that we use in this paper.

Given a training corpus, machine learning algorithms can automatically *learn* classifier functions. The main difference between learning algorithms is the function space in which they search for the classifiers and the measure they use to evaluate how good a classifier is. In our prior work on the applications of machine learning techniques to the premise selection problem [39] we used the SNoW implementation [9] of a multiclass naive Bayes learning method because of its efficiency. In this work, we experiment with state-of-the-art *kernel-based learning methods* for premise selection. We present both methods and show the benefits of using kernels.

4.1 A Naive Bayes Classifier

Naive Bayes is a statistical learning method based on Bayes' theorem about conditional probabilities¹⁰ with a strong (read: naive) independence assumptions. In the naive Bayes setting, the value $C_p(c)$ of the classifier function of a premise p at a conjecture c is the probability that $\mu(c, p) = 1$ given the expressed features $e(c)$.

To understand the difference between the naive Bayes and the kernel-based learning algorithm we need to take a closer look at the naive Bayes classifier. Let θ denote the statement that $\mu(c, p) = 1$ and for each feature $t_i \in T$ let $\tilde{t}_i$ denote that

¹⁰In its simplest form, Bayes' theorem asserts for a probability function P and random variables X and Y that

$$P(X|Y) = \frac{P(Y|X)P(X)}{P(Y)},$$

where $P(X|Y)$ is understood as the conditional probability of X given Y .

$\Phi(c, i) = 1$. Furthermore, let $e(c) = \{s_1, \dots, s_l\} \subseteq T$ be the expressed features of c (with corresponding $\bar{s}_1, \dots, \bar{s}_l$). Then (by Bayes' theorem) we have

$$P(\theta \mid \bar{s}_1, \dots, \bar{s}_l) \propto P(\bar{s}_1, \dots, \bar{s}_l \mid \theta) P(\theta) \quad (1)$$

where the logarithm of the right-hand side can be computed as

$$\ln P(\bar{s}_1, \dots, \bar{s}_l \mid \theta) P(\theta) = \ln P(\bar{s}_1, \dots, \bar{s}_l \mid \theta) + \ln P(\theta) \quad (2)$$

$$= \ln \prod_{i=1}^l P(\bar{s}_i \mid \theta) + \ln P(\theta) \text{ by independence} \quad (3)$$

$$= \sum_{i=1}^m \varphi_i^c \ln P(\bar{t}_i \mid \theta) + \ln P(\theta) \quad (4)$$

$$= \mathbf{w}^T \boldsymbol{\varphi}^c + \ln P(\theta) \quad (5)$$

where

$$w_i := \ln P(\bar{t}_i \mid \theta) \quad (6)$$

There are two things worth noting here. First, $P(\bar{t}_i \mid \theta)$ and $P(\theta)$ might be 0. In that case, taking the natural logarithm would not be defined. In practice, if $P(\bar{t}_i \mid \theta)$ or $P(\theta)$ are 0 the algorithm replaces the 0 with a predefined very small $\varepsilon > 0$.¹¹ Second, line (5) shows that the naive-Bayes classifier is “essentially” (after the monotonic transformation) a linear function of the features of the conjecture. The feature weights $\mathbf{w}$ are computed using formula (6).

4.2 Kernel-based Learning

We saw that the naive Bayes algorithm gives rise to a linear classifier. This leads to several questions: ‘Are there better weights?’ and ‘Can one get better performance with non-linear functions?’. Kernel-based learning provides a framework for investigating such questions. In this subsection we give a simplified, brief description of kernel-based learning that is tailored to our present problem; further information can be found in [5, 25, 28].

4.2.1 Are there Better Weights?

To answer this question we must first define what ‘better’ means. Using the number of problems solved as measure is not feasible because we cannot practically run an ATP for every possible weight combination. Instead, we measure how *good* a classifier approximates our training data. We would like to have that

$$\forall x \in \Gamma : C_p(x) = \mu(x, p).$$

However, this will almost never be the case. To compare how well a classifier approximates the data, we use loss functions and the notion of expected loss that they provide, which we now define.

¹¹For the experiments we used $\ln \varepsilon = -15$.

Definition 4 (Loss function and Expected Loss) A *loss function* is any function $l : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}^+$. Given a loss function l we can then define the *expected loss* $E(\cdot)$ of a classifier C_p as

$$E(C_p) = \sum_{x \in \Gamma} l(C_p(x), \mu(x, p))$$

One might add additional properties such as $l(x, x) = 0$, but this is not necessary. Typical examples of a loss function $l(x, y)$ are the square loss $(y - x)^2$ or the 0-1 loss defined by $I(x = y)$.¹²

We can compare two different classifiers via their expected loss. If the expected loss of classifier C_p is less than the expected loss of a classifier C_q then C_p is the better classifier. It should be noted that a lower expected loss on a particular training set (like the MML proofs) need not necessarily lead to more solved problems by an ATP. One could imagine that the training set contains proofs that are very different from the way a particular ATP would proceed most easily. Also, what happens if the classifier is not able to predict all MML premises, but just a large part of them? These are questions about alternative proofs, and about the robustness of the ATP and prediction methods. An experimental answer is provided in Section 6.3.

4.2.2 Nonlinear Classifiers

It seems straightforward that more complex functions would lead to a lower expected loss and are hence desirable. However, weight optimization becomes tedious once we leave the linear case. Kernels provide a way to use the machinery of linear optimization on non-linear functions.

Definition 5 (Kernel) A *kernel* is a function $k : \Gamma \times \Gamma \rightarrow \mathbb{R}$ satisfying

$$k(x, y) = \langle \phi(x), \phi(y) \rangle$$

where $\phi : \Gamma \rightarrow F$ is a mapping from Γ to an inner product space F with inner product $\langle \cdot, \cdot \rangle$. A kernel can be understood as a *similarity measure* between two entities.

Example 1 A simple kernel for our setting is the linear kernel:

$$k_{\text{lin}}(x, y) := \langle \varphi^x, \varphi^y \rangle$$

with $\langle \cdot, \cdot \rangle$ being the normal dot product in $\mathbb{R}^m$. Here, φ^f denotes the features of a formula f (see Definition 3), and the inner product space F is $\mathbb{R}^m$. A nontrivial example is the Gaussian kernel with parameter σ [7]:

$$k_{\text{gauss}}(x, y) := \exp \left(-\frac{\langle \varphi^x, \varphi^x \rangle - 2\langle \varphi^x, \varphi^y \rangle + \langle \varphi^y, \varphi^y \rangle}{\sigma^2} \right)$$

We can now define our kernel function space in which we will search for classification functions.

¹² I is defined as follows: $I(x = y) = 0$ if $x = y$, and $I(x = y) = 1$ otherwise.

Definition 6 (Kernel Function Space) Given a kernel k , we define

$$\mathcal{F}_k := \left\{ f \in \mathbb{R}^\Gamma \mid f(x) = \sum_{v \in \Gamma} \alpha_v k(x, v), \alpha_v \in \mathbb{R}, \|f\| < \infty \right\}.$$

as our *kernel function space*, where for $f(x) = \sum_{v \in \Gamma} \alpha_v k(x, v)$

$$\|f\| = \sum_{u, v \in \Gamma} \alpha_u \alpha_v k(u, v)$$

Essentially, every function in $\mathcal{F}_k$ compares the input x with formulas in Γ using the kernel, and the weights α determine how important each comparison is.¹³

The kernel function space $\mathcal{F}_k$ naturally depends on the kernel k . It can be shown that when we use k_{lin} , $\mathcal{F}_{k_{\text{lin}}}$ consists of linear functions of the MML features T . In contrast, the Gaussian kernel k_{gauss} gives rise to a nonlinear (in the features) function space.

4.2.3 Putting it All Together

Having defined loss functions, kernels and kernel function spaces we can now define how kernel-based learning algorithms learn classifier functions. Given a kernel k and a loss function l , recall that we measure how good a classifier C_p is with the expected loss $E(C_p)$. With all our definitions it seems reasonable to define C_p as

$$C_p := \arg \min_{f \in \mathcal{F}_k} E(f) \quad (7)$$

However, this is not what a kernel based learning algorithm does. There are two reasons for this. First, the minimum might not exist. Second, in particular when using complex kernel functions, such an approach might lead to overfitting: C_p might perform very well on our training data, but badly on data that was not seen before. To handle both problems, a *regularization parameter* $\lambda > 0$ is introduced to penalize complex functions. This regularization parameter allows us to place a bound on possible solution which together with the fact that $\mathcal{F}_k$ is a Hilbert space ensures the existence of C_p . Hence we define

$$C_p = \arg \min_{f \in \mathcal{F}_k} E(f) + \lambda \|f\|^2 \quad (8)$$

Recall from the definition of $\mathcal{F}_k$ that C_p has the form

$$C_p(x) = \sum_{v \in \Gamma} \alpha_v k(x, v), \quad (9)$$

with $\alpha_v \in \mathbb{R}$. Hence, for any fixed λ , we only need to compute the weights α_v for all $v \in \Gamma$ in order to define C_p . In Section 4.3 we show how to solve this optimization problem in our setting.

¹³ A more general approach to kernel spaces is available [24].

4.2.4 Naive Bayes vs Kernel-based Learning

Kernel-based methods typically outperform the naive Bayes algorithm. There are several reasons for this. Firstly and most importantly, while naive Bayes is essentially a linear classifier, kernel based methods can learn non-linear dependencies when an appropriate non-linear (e.g. Gaussian) kernel function is used. This advantage in expressiveness usually leads to significantly better generalization¹⁴ performance of the algorithm given properly estimated hyperparameters (e.g., the kernel width σ for Gaussian functions). Secondly, kernel-based methods are formulated within the regularization framework that provides mechanism to control the errors on the training set and the complexity (“expressiveness”) of the prediction function. Such setting prevents overfitting of the algorithm and leads to notably better results compared to unregularized methods. Thirdly, some of the kernel-based methods (depending on the loss function) can use very efficient procedures for hyperparameter estimation (e.g. fast leave-one-out cross-validation [22]) and therefore result in a close to optimal model for the classification/regression task. For such reasons kernel-based methods are among the most successful algorithms applied to various problems from bioinformatics to information retrieval to computer vision [28]. A general advantage of naive Bayes over kernel-based algorithms is the computational efficiency, particularly when taking into account the fact that computing the kernel matrix is generally quadratic in the number of training data points. However, recent advances in large scale learning have led to extensions of various kernel-based methods such as support vector machines (SVM), with sublinear complexity, provably fast convergence rate, and the generalization performance that cannot be matched by most of the methods in the field [27].

4.3 MOR Experimental Setup

For our experiments, we will now define a kernel-based multi-output ranking (MOR) algorithm; it straightforwardly extends a previously defined preference learning algorithm [22, 33].

Let $\Gamma = \{x_1, \dots, x_n\}$. Then formula (9) becomes

$$C_p(x) = \sum_{i=1}^n \alpha_i k(x, x_i)$$

Using this and the square-loss $l(x, y) = (x - y)^2$ function, solving (8) is equivalent to finding weights α_i that minimize

$$\min_{\alpha_1, \dots, \alpha_n} \left[\sum_{i=1}^n \left(\sum_{j=1}^n \alpha_j k(x_i, x_j) - \mu(x_i, p) \right)^2 + \lambda \sum_{i,j=1}^n \alpha_i \alpha_j k(x_i, x_j) \right] \quad (10)$$

Recall that C_p is the classifier for a single premise. Since we eventually want to rank all premises, we need to train a classifier for each premise. So we need to

¹⁴Generalization is the ability of a machine learning algorithm to perform accurately on new, unseen examples after training on a finite data set.

find weights $\alpha_{i,p}$ for each premise p . This does seem to complicate the problem quite a bit. However, we can use the fact that for each premise p , C_p depends on the values of $k(x_i, x_j)$, where $1 \leq i, j \leq n$, to speed up the computation. Instead of learning the classifiers C_p for each premise separately, we learn all the weights $\alpha_{p,i}$ simultaneously.

To do this, we first need some definitions. Let

$$A = (\alpha_{i,p})_{i,p} \quad (1 \leq i \leq n, p \in \Gamma).$$

A is the matrix where each column contains the parameters of one premise classifier. Define the kernel matrix K and the label matrix Y as

$$\begin{aligned} K &:= (k(x_i, x_j))_{i,j} \quad (1 \leq i, j \leq n) \\ Y &:= (\mu(x_i, p))_{i,p} \quad (1 \leq i \leq n, p \in \Gamma). \end{aligned}$$

We can now rewrite (10) in matrix notation to state the problem for all premises:

$$\arg \min_A \text{tr}((Y - KA)^T(Y - KA) + \lambda A^T KA) \quad (11)$$

where $\text{tr}(A)$ denotes the trace of the matrix A . Taking the derivative with respect to A leads to:

$$\begin{aligned} \frac{\partial}{\partial A} \text{tr}((Y - KA)^T(Y - KA) + \lambda A^T KA) \\ &= \text{tr}(-2K(Y - KA) + 2\lambda KA) \\ &= \text{tr}(-2KY + (2KK + 2\lambda K)A) \end{aligned}$$

To find the minimum, we set the derivative to zero and solve with respect to A . This leads to:

$$A = (K + \lambda I)^{-1} Y \quad (12)$$

In the experiments, we use the Gaussian kernel k_{gauss} we defined in Example 1. Ergo, if we fix the regularization parameter λ and the kernel parameter σ we can find the optimal weights through simple matrix computations. Thus, to fully determine the classifiers, it remains to find good values for the parameters λ and σ . This is done, as is common with such parameter optimization for kernel methods, by simple (logarithmically scaled) grid search and cross-validation on the training data using a 70/30 split.¹⁵

5 Data: The MPTP2078 Benchmark

The effects of using the finer dependency data (both for direct re-proving and for training premise selection), and the effect of using our kernel-based MOR algorithm

¹⁵For this, we first define a logarithmically scaled set of potential parameters. The training set is then randomly split in two parts cv_{train} and cv_{test} with cv_{train} containing 70 % of the training data and cv_{test} containing the remaining 30 %. For each set of parameters, the algorithm is trained on cv_{train} and evaluated on cv_{test} . The process is repeated 10 times. The set of parameters with the best average performance is then picked for the real evaluation.

are evaluated on a newly created large-theory benchmark¹⁶ of 2078 related MML problems, which extends the older and smaller MPTP Challenge benchmark.

The original MPTP Challenge benchmark was created in 2006, with the purpose of supporting the development of ARLT (automated reasoning for large theories) techniques. It contains 252 related problems, leading to the Mizar proof of one implication of the Bolzano-Weierstrass theorem. The challenge has two divisions: *chainy* (harder) and *bushy* (easier). The motivation behind them is given below when we describe their analogs in the MPTP2078 benchmark.

Both the ARLT techniques and the computing power (particularly multi-core technology) have developed since 2006. Appropriately, we define a larger benchmark with a larger numbers of problems and premises, and making use of the fine-grained dependency analysis. The larger number of problems together with their dependencies more faithfully mirror the setting that mathematicians are facing: typically, they know a number of related theorems and their proofs when solving a new problem.

The new MPTP2078 benchmark is created as follows: The 33 Mizar articles¹⁷ from which problems were previously selected for constructing the MPTP Challenge are used. We however use a new version of Mizar and MML allowing the precise dependency analysis, and use all problems from these articles. This yields 2078 problems. As with the MPTP Challenge benchmark, we create two groups (divisions) of problems.

Chainy: Versions of the 2078 problems containing all previous MML contents as premises. This means that the conjecture is attacked with “all existing knowledge”, without any premise selection. This is a common use case for proving new conjectures fully automatically, see also Section 6.2. In the MPTP Challenge, the name *chainy* has been introduced for this division, because the problems and dependencies are ordered into a chronological chain, emulating the growth of the library.

Bushy: Versions of the 2078 problems with premises pruned using the new fine-grained dependency information (extracted with the *FD* method described in Section 3). This use-case has been introduced in proof assistants by Harrison’s MESON_TACTIC [12], which takes an explicit list of premises from the large library selected by a knowledgeable user, and attempts to prove the conjecture just from these premises. We are interested in how powerful ATPs can get on MML with such precise advice.

To evaluate the benefit of having fine-grained dependencies, we additionally also produce versions of the 2078 problems with premises pruned by the old heuristic dependency-pruning method *D* (see Section 3) used for constructing re-proving problems by the MPTP system.

Table 1 provides a summary of the fine-grained dependency data for the set of 33 Mizar articles coming from the MPTP2078 benchmark. For each theorem in the sequence of the 33 Mizar articles (ordered from first to last by their order in the

¹⁶<http://wiki.mizar.org/twiki/bin/view/Mizar/MpTP2078>

¹⁷<https://raw.githubusercontent.com/JUrban/MPTP2078/master/ArticlesInMMLOrder>. These articles can be explored online at http://mws.cs.ru.nl/~mptp/7.11.07_4.156.1112/html/.

Table 1 Effectiveness of fine-grained dependencies on the 33 MPTP2078 articles ordered from top to bottom by their order in the MML

Article	Theorems	Expl. Deps.	Uniq. Expl. Deps.	Fine Deps. (FD)	MPTP Deps. (D)
xboole_0	7	4	2.7	11.57	12.62
xboole_1	117	5.34	2.3	15.27	17.86
enumset1	87	3.26	2.7	10.67	10.82
zfmisc_1	129	4.74	2.9	16.59	21.08
subset_1	43	4.62	2.4	22.30	28.15
setfam_1	48	6.56	2.4	25.62	37.93
relat_1	184	5.66	2.2	19.97	27.31
funct_1	107	7.69	3.4	22.94	42.05
ordinal1	37	7.81	3.9	26	61.92
wellord1	53	11.7	5.6	30.45	55.5
relset_1	32	4.71	2.6	27	55.59
mcart_1	92	5.71	2.8	21.25	29.77
wellord2	24	14.2	6.9	36.41	75.2
funct_2	124	4.14	2.5	30.77	92.45
finset_1	15	8.66	3.9	23.93	72.22
pre_topc	36	6.47	3.2	35.58	53.51
orders_2	56	10.6	5.3	46.28	79.77
lattices	27	5.59	3.1	43	72.96
tops_1	71	6.67	4.2	43.46	66.42
tops_2	65	7.36	4.0	37.41	103.4
compts_1	23	17.7	8.6	48.86	102.9
connsp_2	29	9.86	6.4	39.51	96.5
filter_1	61	14.6	5.3	52.27	122.8
lattice3	55	8.6	4.2	47.07	92.25
yellow_0	70	6.75	3.2	26.55	46.12
yellow_1	28	9.03	5.1	53.17	128.3
waybel_0	76	9.34	4.1	35.03	82.34
tmap_1	141	8.78	4.5	47.04	140.0
tex_2	74	7.66	5.0	37.31	155.6
yellow_6	44	11.2	6.4	50.31	173.5
waybel_7	46	13.0	7.7	57.10	140.3
waybel_9	41	9	5.2	51.56	156.5
yellow19	36	8.44	5.3	53.02	137.2

Article: Mizar Article relevant to the MPTP2078 benchmark

Theorems: Total number of theorems in the article

Expl. Deps.: Average number of (non-unique) explicit dependencies (to theorems, definitional theorems, and schemes) per theorem in the article

Uniq. Expl. Deps.: Average number of unique explicit dependencies per theorem

Fine Deps.: Average number of all (both explicitly and implicitly) used items per theorem as computed by dependency analysis (method *FD*)

MPTP Deps.: Average number of items per theorem computed by the MPTP fixpoint algorithm (method *D*)

MML) we show the average number of explicit dependencies involved in their proofs and the average number of its fine-grained dependencies computed by *FD* and *D*. All three datasets contain the same conjectures. They only differ in the number of redundant axioms. The average number of premises is 1,976.5 for the unpruned (chainy) problems, 74 for the bushy-old problems (*D*) and 31.5 for bushy problems (*FD*). Table 2 summarizes the datasets.

Table 2 Average number of premises in the three datasets

Dataset	Premises used	Avg. number of premises
Chainy	All previous	1,976.5
Bushy-old	MPTP dependencies D	74
Bushy	Fine-grained dependencies FD	31.5

6 Experiments and Results

We use Vampire 0.6 [20] as the ATP system for all experiments conducted here. Adding other ATP systems is useful [36], and there are metasearchers like MaLAREa which attempt to exploit the joint power of different systems in an organized way. However, the focus of this work is on premise selection, which has been shown to have similar effect across the main state-of-the-art ATP systems. Another reason for using the recent Vampire is that in [36], Vampire with the SInE preprocessor was sufficiently tested and tuned on the MML data, providing a good baseline for comparing learning-based premise-selection methods with robust state-of-the-art methods that can run on any isolated large problem without any learning. All measurements are done on an Intel Xeon E5520 2.27GHz server with 8 GB RAM and 8 MB CPU cache. Each problem is always assigned one CPU, which only belongs to the problem during its execution.

In Section 6.1 we evaluate the ATP performance when fine-grained dependencies (bushy problems, FD -method) are used by comparing it to the ATP performance on the old MPTP heuristic pruning (bushy-old problems, D -method), and to the ATP performance on the large (chainy) versions of the MPTP2078 problems. These results show that there is a lot to gain by constructing good algorithms for premise selection. In Section 6.2 SNoW's naive Bayes and the MOR machine learning algorithms are incrementally trained on the fine-grained MML dependency data, and their precision in predicting the MML premises on new problems are compared. This standard machine-learning comparison is then in Section 6.3 completed by running Vampire on the premises predicted by the MOR and SNoW algorithms. This provides information about the overall theorem-proving performance of the whole dependency-computing/learning/ATP stack. This performance is compared to the performance of Vampire/SInE.

6.1 Using the Fine-Grained Dependency Analysis for Re-proving

The first experiment evaluates the effect of fine-grained dependencies on re-proving Mizar theorems automatically. The results of Vampire/SInE run with 10s time limit¹⁸ on the datasets defined above are shown in Table 3.

¹⁸There are several reasons why we use low time limits instead of the high time limits used in the CASC competition. First, Vampire performs reasonably with low time limits [36]. It is clear that much more computation can be done within 10 seconds in 2011, than fifteen years ago, when CASC started. Second, low time limits are useful when running many experiments and when combining different strategies. Third, in typical ITP proof-advice scenarios [37], the preferable query response time is in (tens of) seconds.

Table 3 Performance of Vampire (10 s time limit) on 2078 MPTP2078 benchmark with different axiom pruning

Pruning	Solved problems	Solved as percentage
Chainy	548	26.4
Chainy (Vampire -d1)	556	26.8
Bushy-old (<i>D</i> method)	1,023	49.2
Bushy (<i>FD</i> method)	1,105	53.2

Vampire (run in the unmodified automated CASC mode) solves 548 of the unpruned problems. If we use the -d1 parameter,¹⁹ Vampire solves 556 problems. Things change a lot with external premise pruning. Vampire solves 1023 of the 2078 problems when the old MPTP heuristic pruning (bushy-old, *D*-method) is applied. Using the pruning based on the new fine-grained analysis (*FD*-method) Vampire solves 1105 problems, which is an 8 % improvement over the heuristic pruning in the number of problems solved. Since the heuristic pruning becomes more and more inaccurate as the MML grows (the ratio of MPTP Deps. to Fine Deps. in Table 1 has a growing trend from top to bottom), we can conjecture that this improvement will be even more significant when considering the whole MML. Also note that these numbers point to the significant improvement potential that can be gained by good premise selection: the performance on the pruned dataset is doubled in comparison to the unpruned dataset. Again, this ratio grows as MML grows, and the number of premises approaches 100.000.²⁰

6.2 Combining Fine-Grained Dependencies with Learning

For the next experiment, we emulate the growth of the library (limited to the 2078 problems), by considering all previous theorems and definitions when a new conjecture is attempted. This is a natural “ATP advice over the whole library” scenario, in which the ATP problems however become very large, containing thousands of the previously proved formulas. Premise selection can therefore help significantly.

For each problem we use the fine-grained MML dependencies extracted from all previous proofs²¹ to train the premise-selection algorithms, evaluate their advice on the current problem, and compare the recall (and also the ATP performance in the next subsection). This corresponds to the situation in general mathematics when mathematicians not only know many previous theorems, but also re-use previous problem solving knowledge. This approach requires us to do 2078 training/evaluation steps as the problems and their proofs are added to the library and the dataset grows. We compare the MOR algorithm with SNoW’s naive Bayes.

¹⁹The -d parameter limits the depth of recursion for the SInE algorithm. Running Vampire with the -d1 pruning parameter resulted in significant performance improvement on large Mizar problems [36].

²⁰On the whole MML with Vampire/SInE, this ratio is 39 % to 14 % [36].

²¹We do not evaluate the performance of learning on the approximate bushy-old dependencies here and in the next subsection. Table 1 and Section 6.1 already sufficiently show that these data are less precise than the fine-grained MML dependencies.

Figure 1 shows the average recall of SNoW and MOR on this dataset. Recall measures how well the predictions correspond to the training data. Formally, the recall $recall(c, n)$ for one conjecture c when n premises are advised is defined as:

$$recall(c, n) = \frac{|usedPremises(c) \cap advisedPremises(c, n)|}{|usedPremises(c)|}$$

This means that the segments of n best-ranked premises obtained from the algorithms are compared with the actual premises used in the MML proof, and for each n we compute the average (relative) size of the overlap. The size of the segment (n) is the x-axis in Fig. 1. It can be seen that the MOR algorithm fits the training data significantly better than SNoW. E. g., on average 88 % of the used premises are within the 50 highest MOR-ranked premises, whereas when we consider the SNoW ranking only around 80 % of the used premises are with the 50 highest ranked premises.

Recall is a fast method for comparing two different premise selection algorithms. This kind of comparison is the standard endpoint in machine learning applications like keyword-based document retrieval, consumer choice prediction, etc. However, in a semantic domain like ours, we can go further, and see how this improved prediction performance helps the theorem proving process.

It should be noted that a recall value of < 100 does not necessarily imply that the ATP cannot prove the conjecture from the predicted premises since our definition of used premises depends on the Mizar proof of the conjecture. Usually, there are many other proofs, using a subset of or even completely different premises than the premises used in the Mizar proof. ATPs in particular often find proofs using different premises [3].

6.3 Combining it All: ATP Supported by Learning from Fine Dependencies

In the last experiment, we finally chain the whole ITP/Learning/ATP stack together, and evaluate how the improved premise selection is reflected in the performance of

Fig. 1 Comparison of the SNoW and MOR average recall of premises used in the Mizar proofs. The x-axis shows the number of premises asked from SNoW and MOR, and the y-axis shows their relative overlap with the premises used in the original Mizar proof

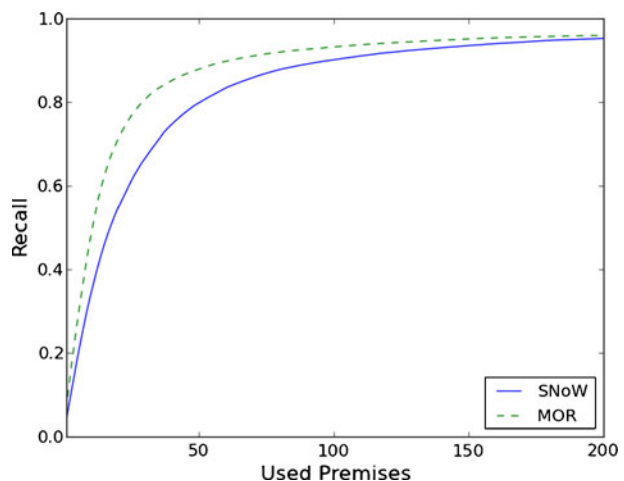
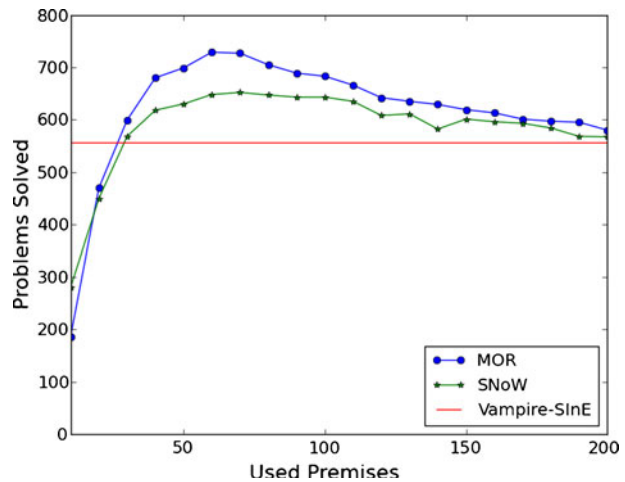


Fig. 2 Comparison of the number of solved problems by SNoW and MOR. The x-axis shows the number of premises given to Vampire, and the y-axis shows the number of problems solved within 5 s. The number of problems solved by Vampire/SInE in 10 seconds is given as a baseline



automated theorem proving on new large-theory conjectures. Both the naive Bayes (SNoW) and the new MOR learning algorithms are evaluated.

Figure 2 shows the numbers of problems solved by Vampire using different numbers of the top premises predicted by SNoW and MOR, and a 5 s time limit. The maximum number of problems solved with MOR is 729 with the top 60 advised premises (MOR-60). SNoW's maximum is 652 with the top 70 premises (SNoW-70). The corresponding numbers for a 10 second time limit are 795 solved problems for MOR-60, and 722 for SNoW-70. Table 4 compares these data with the overall performance of Vampire with a 10 s time limit run on problems with pruning done by SInE. Using the SNoW-60 resp. MOR-70 advising methods results in a 32 % resp. 45 % improvement over the 548 problems Vampire solves in auto-mode, and in a 30 % resp. 43 % improvement over the 556 problems solved by Vampire using the -d1 option.

Table 5 compares the performance of Vampire/SInE with the performance of SNoW and MOR when, for each of algorithm, we take the union of the two 5 s runs with the largest joint coverage. Those are obtained by using the top 40 advised premises and the top 180 advised premises for SNoW, and the top 40 and top 100 advised premises for MOR. These SNoW resp. MOR combined runs give a 44 % resp. 50 % improvement over the 548 problems Vampire solves in auto-mode, and a 42 % resp. 48 % improvement over the 556 problems solved by Vampire using the -d1 option.

Note that Vampire/SInE does strategy scheduling internally, and with different SInE parameters. Thus combining two different premise selection strategies is

Table 4 Comparison of Vampire (10 s time limit) performance on MPTP2078 with different premise selections

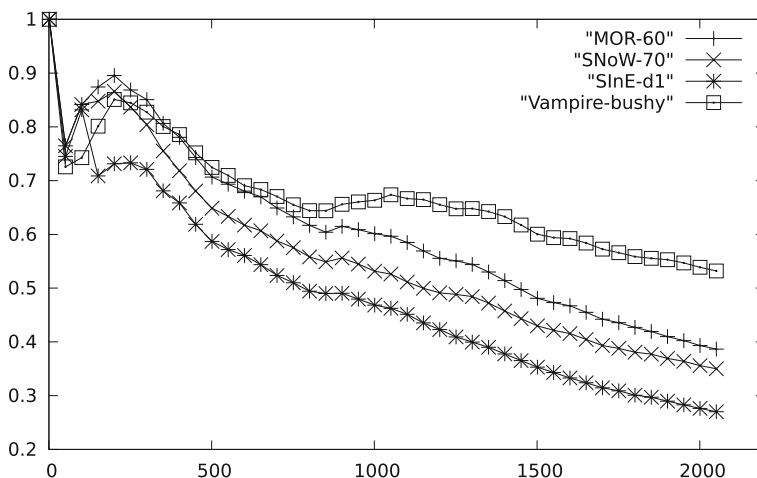
System	Solved problems	Gain over Vampire (%)
Vampire/SInE	548	0
Vampire/SInE (-d1)	556	1.5
SNoW-70	722	31.8
MOR-60	795	45.1

Table 5 10s performance of the two strategies with the largest joint coverage for SNoW and MOR

System	Solved problems	Gain over Vampire (%)
Vampire/SInE	548	0
Vampire/SInE (-d1)	556	1.5
SNoW-40/180	788	43.7
MOR-40/100	824	50.4

perfectly comparable to the way Vampire's automated mode is constructed and used. Also note that combining the two different ways in which unadvised Vampire/SInE was run is not productive: the union of both unadvised runs is just 559 problems, which is only 3 more solved problems (generally in 20 s) than with running Vampire/SInE with -d1 for 10 s.

Finally, Figs. 3 and 4 compare the cumulative and average performance of the algorithms (combined with ATPs) at different points of the MPTP2078 benchmark, using the chronological ordering of the MPTP2078 problems. The average available number of premises for the theorems ordered chronologically grows linearly (the earlier theorems and definitions become eligible premises for the later ones), making the later problems harder on average. Figure 3 shows the percentage of problems solved on initial segments of problems using a step value of 50. The last value (for 2050) corresponds to the percentage of problems solved on the whole MPTP2078 (0.38 for MOR-60), while for example the value for 1000 (0.60 for MOR-60) shows the percentage of problems solved on the first 1000 MPTP2078 problems. Figure 4 compares the number of solved problems of the algorithms when the problems are divided into four successive segments of equal size. Note that even with the precise use of the MML premises the problems do not have uniform difficulty across the benchmark, and on average, even the bushy versions of the later problems get harder. To visualize this, we also add the values for Vampire-bushy to the comparison.

**Fig. 3** Percentage of problems solved in 10 s on initial segments of MPTP2078

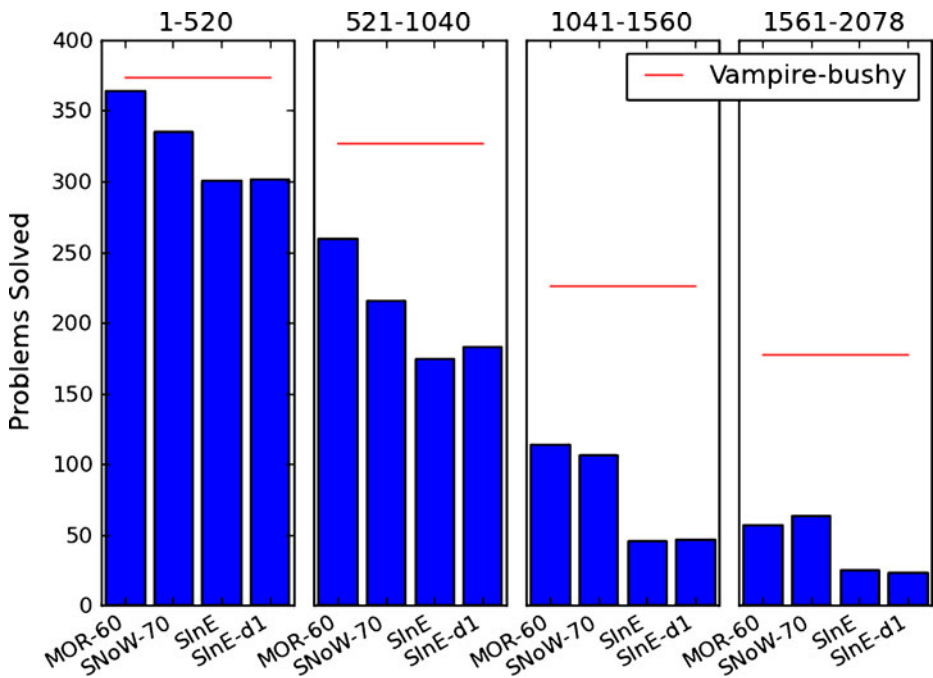


Fig. 4 Number of solved problems in 10 s in four successive equally sized segments of MPTP2078

Except from small deviations, the ratio of solved problems decreases for all the algorithms. Vampire/MOR-60 is able to keep up with Vampire-bushy in the range of the initial 800 problems, and after that the human selection increasingly outperforms all the algorithms. Making this gap as small as possible is an obvious challenge on the path to strong automated reasoning in general mathematics.

7 Conclusion and Future Work

The performance of automated theorem proving over real-world mathematics has been significantly improved by using detailed formally-assisted analysis of a large corpus of theorems and proofs, and by using improved prediction algorithms. In particular, premise selection based on learning from fine-grained proof dependencies improves ATP performance in large mathematical theories by about 44 % when using off-the-shelf learning methods like naive Bayes in comparison with state-of-the-art general premise-selection heuristics like SInE. It was shown that this can be further improved to about 50 % when employing state-of-the-art kernel-based learning methods.

Automated reasoning in large mathematical libraries is becoming a complex AI field, allowing interplay of very different AI techniques. Manual tuning of strategies and heuristics does not scale to large complicated domains, and data-driven approaches are becoming very useful in handling such domains. At the same time, existing strong learning methods are typically developed on imprecise domains,

where feedback loops between prediction and automated verified confirmation as done for example in MaLAREa are not possible. The stronger such AI systems become, the closer we get to formally assisted mathematics, both in its “forward” (constructing new formal proofs) and “reverse” (analysis of existing proofs) form. We thus encounter another positive feedback loop for further study: the larger the body of formally expressed and verified ideas, the smarter the AI systems that learn from them.

While we have achieved 50 % ATP improvement on large problems by better premise selection resulting in 824 problems proved within 10 seconds, we know (from Section 6.1) that with a better premise selection it is possible to prove at least 1105 problems. Thus, there is still a great opportunity for improved premise selection algorithms. One interesting challenge is finding good features for the formal mathematical domain. So far, we have been using only the symbols and terms occurring in formulas as their feature characterizations, but other features are possible, and very likely used by mathematicians. In particular, for ad hoc problem collections like the TPTP library, where symbols are used inconsistently across different problems, formula features that abstract from particular symbols will likely be needed. An interesting AI problem that is becoming more relevant as the ATP methods for mathematics are getting stronger, is translation of the (typically resolution-based) ATP proofs into human-understandable [10, 23] formats used by mathematicians. Machine learning from large human-proof corpora like MML is likely to be useful for this task, in a similar way to how it is useful for finding relevant premises.

Acknowledgements We would like to thank the anonymous JAR referees for a number of questions, comments, and insights that helped to significantly improve the final version of this paper.

References

1. Alama, J.: Formal proofs and refutations. Ph.D. thesis, Stanford University (2009)
2. Alama, J., Brink, K., Mamane, L., Urban, J.: Large formal wikis: Issues and solutions. In: Davenport, J.H., Farmer, W.M., Urban, J., Rabe, F. (eds.) *Calculus/MKM. Lecture Notes in Computer Science*, vol. 6824, pp. 133–148. Springer (2011)
3. Alama, J., Kühlwein, D., Urban, J.: Automated and human proofs in general mathematics: an initial comparison. In: Bjørner, N., Voronkov, A. (eds.) *LPAR. Lecture Notes in Computer Science*, vol. 7180, pp. 37–45. Springer (2012)
4. Alama, J., Mamane, L., Urban, J.: Dependencies in formal mathematics: applications and extraction for Coq and Mizar. In: Jeuring, J., Campbell, J.A., Carette, J., Reis, G.D., Sojka, P., Wenzel, M., Sorge, V. (eds.) *AISC/MKM/Calculus. Lecture Notes in Computer Science*, vol. 7362, pp. 1–16. Springer (2012)
5. Aronszajn, N.: Theory of reproducing kernels. *Trans. Am. Math. Soc.* **68**, 337–404 (1950)
6. Bertot, Y., Castéran, P.: Interactive theorem proving and program development. *Coq’Art: the calculus of inductive constructions*. In: *Texts in Theoretical Computer Science*. Springer (2004)
7. Bishop, C.M.: *Pattern Recognition and Machine Learning (Information Science and Statistics)*. Springer, Secaucus (2006)
8. Blanchette, J.C., Bulwahn, L., Nipkow, T.: Automatic proof and disproof in Isabelle/HOL. In: Tinelli, C., Sofronie-Stokkermans, V. (eds.) *FroCos. Lecture Notes in Computer Science*, vol. 6989, pp. 12–27. Springer (2011)
9. Carlson, A., Cumby, C., Rosen, J., Roth, D.: The SNoW learning architecture. Tech. Rep. UIUCDCS-R-99-2101, UIUC Computer Science Department (1999). <http://cogcomp.cs.illinois.edu/papers/CCRR99.pdf>
10. Davis, M.: Obvious logical inferences. In: Hayes, P.J. (ed.) *IJCAI*, pp. 530–531. Kaufmann (1981)

11. Grabowski, A., Kornilowicz, A., Naumowicz, A.: Mizar in a nutshell. *J. Formaliz. Reason.* **3**(2), 153–245 (2010)
12. Harrison, J.: HOL light: A tutorial introduction. In: Srivas, M.K., Camilleri, A.J. (eds.) *FMCAD. Lecture Notes in Computer Science*, vol. 1166, pp. 265–269. Springer (1996)
13. Harrison, J., Slind, K., Arthan, R.: HOL. In: Wiedijk, F. (ed.) *The Seventeen Provers of the World. Lecture Notes in Computer Science*, vol. 3600, pp. 11–19. Springer (2006)
14. Hoder, K., Voronkov, A.: Sine qua non for large theory reasoning. In: Bjørner, N., Sofronie-Stokkermans, V. (eds.) *CADE. Lecture Notes in Computer Science*, vol. 6803, pp. 299–314. Springer (2011)
15. Meng, J., Paulson, L.C.: Translating higher-order clauses to first-order clauses. *J. Autom. Reason.* **40**(1), 35–60 (2008)
16. de Moura, L.M., Bjørner, N.: Z3: An efficient SMT solver. In: Ramakrishnan, C.R., Rehof, J. (eds.) *TACAS. Lecture Notes in Computer Science*, vol. 4963, pp. 337–340. Springer (2008)
17. Nipkow, T., Paulson, L.C., Wenzel, M.: Isabelle/HOL—A proof assistant for higher-order logic. *Lecture Notes in Computer Science*, vol. 2283. Springer (2002)
18. Paulson, L.C., Susanto, K.W.: Source-level proof reconstruction for interactive theorem proving. In: Schneider, K., Brandt, J. (eds.) *TPHOLs. Lecture Notes in Computer Science*, vol. 4732, pp. 232–245. Springer (2007)
19. Pease, A., Sutcliffe, G.: First order reasoning on a large ontology. In: Sutcliffe, G., Urban, J., Schulz, S. (eds.) *Proceedings of the CADE-21 Workshop on Empirically Successful Automated Reasoning in Large Theories*, Bremen, Germany, 17th July 2007. *CEUR Workshop Proceedings*, vol. 257. CEUR-WS.org (2007)
20. Riazanov, A., Voronkov, A.: The design and implementation of VAMPIRE. *AI Commun.* **15**(2–3), 91–110 (2002)
21. Richard, M.D., Lippmann, R.P.: Neural network classifiers estimate Bayesian a posteriori probabilities. *Neural Comput.* **3**(4), 461–483 (2010)
22. Rifkin, R., Yeo, G., Poggio, T.: Regularized least-squares classification. In: Suykens, J., Horvath, G., Basu, S., Michelli, C., Vandewalle, J. (eds.) *Advances in Learning Theory: Methods, Model and Applications*, pp. 131–154. IOS Press, Amsterdam (2003)
23. Rudnicki, P.: Obvious inferences. *J. Autom. Reason.* **3**(4), 383–393 (1987)
24. Schoelkopf, B., Herbrich, R., Williamson, R., Smola, A.J.: A generalized representer theorem. In: Helmbold, D., Williamson, R. (eds.) *Proceedings of the 14th Annual Conference on Computational Learning Theory*, pp. 416–426. Berlin, Germany (2001)
25. Scholkopf, B., Smola, A.J.: *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press, Cambridge (2001)
26. Schulz, S.: E - A brainiac theorem prover. *AI Commun.* **15**(2–3), 111–126 (2002)
27. Shalev-Shwartz, S., Singer, Y., Srebro, N., Cotter, A.: Pegasos: primal estimated sub-gradient solver for SVM. *Math. Program.* **127**(1), 3–30 (2011)
28. Shawe-Taylor, J., Cristianini, N.: *Kernel Methods for Pattern Analysis*. Cambridge University Press, New York (2004)
29. Simpson, S.G.: *Subsystems of Second Order Arithmetic*, 2nd edn. *Perspectives in Mathematical Logic*. Springer (2009)
30. Solovay, R.: AC and strongly inaccessible cardinals. Available on the Foundations of Mathematics archives at <http://www.cs.nyu.edu/pipermail/fom/2008-March/012783.html> (2008)
31. Tarski, A.: On well-ordered subsets of any set. *Fundam. Math.* **32**, 176–183 (1939)
32. Trybulec, A.: Tarski Grothendieck set theory. *Formaliz. Math.* **1**(1), 9–11 (1990)
33. Tsvitsivadze, E., Pahikkala, T., Boberg, J., Salakoski, T., Heskes, T.: Co-regularized least-squares for label ranking. In: Fürnkranz, J., Hüllermeier, E. (eds.) *Preference Learning*, pp. 107–123. Springer, Berlin (2011)
34. Urban, J.: MPTP—motivation, implementation, first experiments. *J. Autom. Reason.* **33**(3–4), 319–339 (2004)
35. Urban, J.: MPTP 0.2: design, implementation, and initial experiments. *J. Autom. Reason.* **37**(1–2), 21–43 (2006)
36. Urban, J., Hoder, K., Voronkov, A.: Evaluation of automated theorem proving on the Mizar Mathematical Library. In: Fukuda, K., van der Hoeven, J., Joswig, M., Takayama, N. (eds.) *ICMS. Lecture Notes in Computer Science*, vol. 6327, pp. 155–166. Springer (2010)
37. Urban, J., Rudnicki, P., Sutcliffe, G.: ATP and presentation service for Mizar formalizations. *J. Autom. Reason.* **50**, 229–241 (2013)
38. Urban, J., Sutcliffe, G.: Automated reasoning and presentation support for formalizing mathematics in Mizar. In: Autexier, S., Calmet, J., Delahaye, D., Ion, P.D.F., Rideau, L., Rioboo,

- R., Sexton, A.P. (eds.) AISC/MKM/Calculus. Lecture Notes in Computer Science, vol. 6167, pp. 132–146. Springer (2010)
39. Urban, J., Sutcliffe, G., Pudlák, P., Vyskocil, J.: MaLAREa SG1- machine learner for automated reasoning with semantic guidance. In: Armando, A., Baumgartner, P., Dowek, G. (eds.) IJCAR. Lecture Notes in Computer Science, vol. 5195, pp. 441–456. Springer (2008)
40. Weidenbach, C., Dimova, D., Fietzke, A., Kumar, R., Suda, M., Wischniewski, P.: SPASS version 3.5. In: Schmidt, R.A. (ed.) CADE. LNCS, vol. 5663, pp. 140–145. Springer (2009)